TABLE 3.1
Comparing agile techniques

Scrum pros	The product owner sets priorities. The team owns decision making. Documentation is lightweight. It supports frequent updating.
Scrum cons	It is difficult to control the cost of changes. It may not be suitable for large teams. It requires expert team members.
XP pros	It emphasizes customer involvement. It establishes rational plans and schedules. There is high developer commitment to the project. There is reduced likelihood of product rejection.
XP cons	There is temptation to "ship" a prototype. It requires frequent meetings about increasing costs. It may allow for excessive changes. There is a dependence on highly skilled team members.
Kanban pros	It has lower budget and time requirements. It allows for early product delivery. Process policies are written down. There is continuous process improvement.
Kanban cons	Team collaboration skills determine success. Poor business analysis can doom the project. Flexibility can cause developers to lose focus. Developer reluctance to use measurement.
DevOps pros	There is reduced time to code deployment. The team has developers and operations staff. The team has end-to-end project ownership. There is proactive monitoring of deployed product.
DevOps cons	There is pressure to work on both old and new code. There is heavy reliance on automated tools to be effective. Deployment may affect the production environment. It requires an expert development team.

Extreme programming (XP) is organized around four framework activities—**planning, design, coding,** and **testing**—XP suggests a number of innovative and powerful techniques that allow an agile team to create frequent software releases that deliver features and functionality that have been described and then prioritized by stakeholders. There is nothing preventing them from using DevOps techniques to decrease their time to deployment.

PROBLEMS AND POINTS TO PONDER

3.1. Read the "Manifesto of Agile Software Development" [Bec01] noted at the beginning of this chapter. Can you think of a situation in which one or more of the four "values" could get a software team into trouble?

3.2. Describe agility (for software projects) in your own words.

3.3. Why does an iterative process make it easier to manage change? Is every agile process discussed in this chapter iterative? Is it possible to complete a project in just one iteration and still be agile? Explain your answers.

3.4. Try to come up with one more "agility principle" that would help a software engineering team become even more maneuverable.

3.5. Why do requirements change so much? After all, don't people know what they want?

3.6. Most agile process models recommend face-to-face communication. Yet today, members of a software team and their customers may be geographically separated from one another. Do you think this implies that geographical separation is something to avoid? Can you think of ways to overcome this problem?

3.7. Write a user story that describes the "favorite places" or "favorites" feature available on most Web browsers.

3.8. Describe the XP concepts of refactoring and pair programming in your own words.

Design element: Quick Look icon magnifying glass: © Roger Pressman

CHAPTER

4 RECOMMENDED PROCESS MODEL

In Chapters 2 and 3, we provided brief descriptions of several software process models and software engineering frameworks. Every project is different, and every team is different. There is no single software engineering framework that is appropriate for every software product. In this chapter, we'll share our thoughts on using an adaptable process that can be tailored to fit the needs of software developers working on many types of products.

KEY CONCEPTS

defining requirements 57
estimating resources 60
evaluating prototype. 64
go, no-go decision. 65
maintain. 69
preliminary architectural design 59
prototype construction 61
prototype evolution. 67
release candidate 68
risk assessment. 66
scope definition. 67
test and evaluate. 63

QUICK LOOK

What is it? Every software product needs a "road map" or "generic software process" of some kind. It doesn't have to be complete before you start, but you need to know where you're headed before you begin. Any road map or generic process should be based on best industry practices.

Who does it? Software engineers and their product stakeholders collaborate to adapt a generic software process model to meet the needs of team and then either follow it directly, or more likely, adapt as needed. Every software team should be disciplined but flexible and self-empowered when needed.

Why is it important? Software development can easily become chaotic without the control and organization offered by a defined process. As we stated in Chapter 3, a modern software engineering approach must be "agile" and embrace changes that are needed to satisfy the stakeholders' requirements. It is important not to be too focused on documents and rituals. The process should only include those activities, controls, and work products that are appropriate for the project team and the product that is to be produced.

What are the steps? Even if a generic process must be adapted to meet the needs of the specific products being built, you need to be sure that all stakeholders have a role to play in defining, building, and testing the evolving software. There is likely to be substantial overlap among the basic framework activities (communication, planning, modeling, construction, and deployment). *Design a little, build a little, test a little, repeat* is a better approach than creating rigid project plans and documents for most software projects.

What is the work product? From the point of view of a software team, the work products are working program increments, useful documents, and data that are produced by the process activities.

How do I ensure that I've done it right? The timeliness, levels of stakeholder satisfaction, overall quality, and long-term viability of the product increments built are the best indicators that your process is working.

A paper by Rajagoplan [Raj14] reviews the general weaknesses of prescriptive software life-cycle approaches (e.g., the waterfall model) and contains several suggestions that should be considered when organizing a modern software development project.

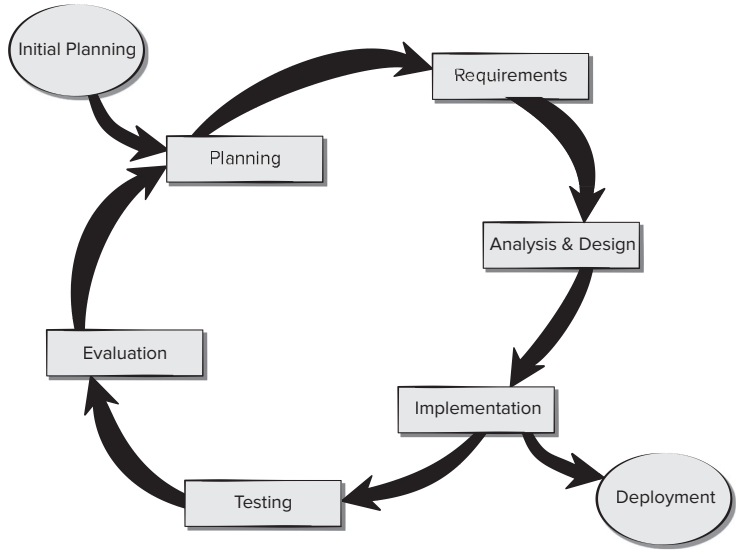
1. It is risky to use a linear process model without ample feedback.
2. It is never possible nor desirable to plan big up-front requirements gathering.
3. Up-front requirements gathering may not reduce costs or prevent time slippage.
4. Appropriate project management is integral to software development.
5. Documents should evolve with the software and should not delay the start of construction.
6. Involve stakeholders early and frequently in the development process.
7. Testers need to become involved in the process prior to software construction.

In Section 2.6, we listed the pros and cons of several prescriptive process models. The waterfall model is not amenable to changes that may need to be introduced once developers start coding. Stakeholder feedback is therefore limited to the beginning and end of the project. Part of the reason for this is the waterfall model suggests that all analysis and design work products be completed before any programming or testing occurs. This makes it hard to adapt to projects with evolving requirements.

One temptation is to switch to an incremental model (Figure 4.1) like the prototyping model or Scrum. Incremental process models involve customers early and often and therefore reduce the risk of creating product that is not accepted by the customers. There is a temptation to encourage lots of changes as stakeholders view each prototype and realize that functions and features they now realize they need are missing. Often, developers do not plan for prototype evolution and create throwaway prototypes.

FIGURE 4.1

Incremental model for prototype development



Recall that the goal of software engineering is to reduce unnecessary effort, so prototypes need to be designed with reuse in mind. Incremental models do provide a better basis for creating an adaptable process if changes can be managed wisely.

In Section 3.5, we discussed the pros and cons of several agile process models other than Scrum. Agile process models are very good at accommodating the uncertain knowledge about the stakeholders' real needs and problems. Key characteristics of agile process models are:

- Prototypes created are designed to be extended in future software increments.
- Stakeholders are involved throughout the development process.
- Documentation requirements are lightweight, and documentation should evolve along with the software.
- Testing is planned and executed early.

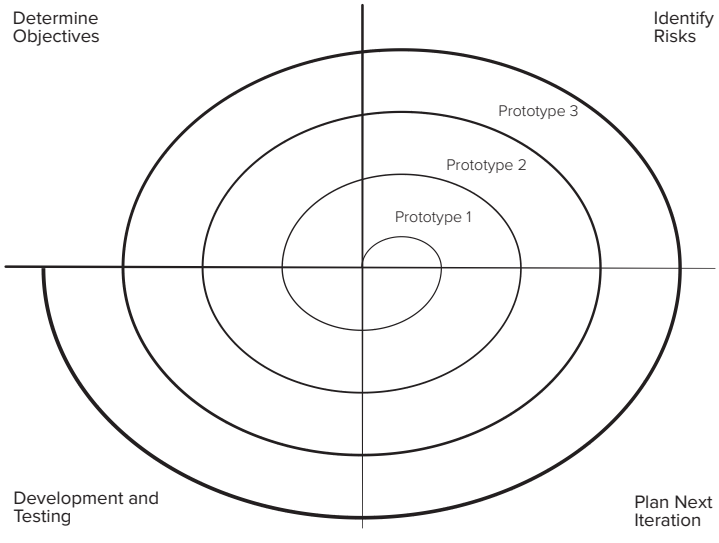
Scrum and Kanban extend these characteristics. Scrum is sometimes criticized for requiring too many meetings. But daily meetings make it hard for developers to stray too far from building products that stakeholders find useful. Kanban (Section 3.5.2) provides a good lightweight tracking system for managing the status and priorities of user stories.

Both Scrum and Kanban allow for controlled introduction of new requirements (user stories). Agile teams are small by design and may not be suitable for projects requiring large numbers of developers, unless the project can be partitioned into small and independently assignable components. Still, agile process models offer many good features that can be incorporated into an adaptable process model.

The spiral model (Figure 4.2) can be thought of as an evolutionary prototyping model with a risk assessment element. The spiral model relies on moderate stakeholder

FIGURE 4.2

Spiral model for prototype development



involvement and was designed for large teams and large projects. Its goal is to create extensible prototypes each time the process is iterated. Early testing is essential. Documentation evolves with the creation of each new prototype. The spiral model is somewhat unique in that formal risk assessment is built in and used as the basis for deciding whether to invest the resources needed to create the next prototype. Some people argue that it may be hard to manage a project using the spiral model, because the project scope may not be known at the start of the project. This is typical of most incremental process models. The spiral is a good basis for building an adaptable process model.

How do agile process models compare to evolutionary models? We've summarized some of the key characteristics in a sidebar.

Characteristics of Agile Models

Agile

1. Not suitable for large high-risk or mission critical projects.
2. Minimal rules and minimal documentation
3. Continuous involvement of testers
4. Easy to accommodate product changes
5. Depends heavily on stakeholder interaction
6. Easy to manage
7. Early delivery of partial solutions
8. Informal risk management
9. Built-in continuous process improvement

Spiral

1. Not suitable for small, low-risk projects
2. Several steps required, along with documentation done up front
3. Early involvement of testers (might be done by outside team)
4. Hard to accommodate product changes until prototype completed
5. Continuous stakeholder involvement in planning and risk assessment
6. Requires formal project management and coordination
7. Project end not always obvious
8. Good risk management
9. Process improvement handled at end of project

Creative, knowledgeable people perform software engineering. They adapt software processes to make them appropriate for the products that they build and to meet the demands of the marketplace. We think that using a spiral-like approach that has agility built into every cycle is a good place to start for many software projects. Developers learn many things as they proceed in the development process. That's why it is important for developers to be able to adapt their process as quickly as practical to accommodate this new knowledge.

4.1 Requirements Definition

Every software project begins with the team trying to understand the problem to be solved and determining what outcomes are important to the stakeholders. This includes understanding the business needs motivating the project and the technical issues which constrain it. This process is called *requirements engineering* and will be discussed in